

Exercise 7.2 — Multi-Env Layout and GitHub Environment Promotion

Course: Optimizaciones y Desempeño — Cloud Deployment Automation

Session: 7 — June 4, 2026

Time allowed: 30 minutes

Submission: Initialize a new repository called oyd-exercise-7-2 and commit/push everything into it. Submit the repository URL only.

Context

You are given a stub repository with a working single-job terraform-cd.yml that runs fmt, validate, and plan in sequence. The pipeline passes, but a reviewer cannot tell which specific check failed when something goes wrong — all three steps live inside one job and report as a single status check.

Your task is to evolve this pipeline into a production-grade workflow that reports each validation step individually on pull requests, uploads the plan as an artifact, and gates promotion from dev to staging behind a required reviewer approval.

Copy the files below into your repository exactly as shown. Do not modify any Terraform file — only .github/workflows/terraform-cd.yml is yours to rewrite.

infra/provider.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  backend "s3" {}
}

provider "aws" {
  region = var.aws_region
}
```

infra/main.tf

```
resource "aws_sqs_queue" "main" {
  name                      = var.queue_name
  visibility_timeout_seconds = var.visibility_timeout_seconds
}
```

infra/variables.tf

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "queue_name" {
  description = "Name of the SQS queue"
  type        = string
}

variable "visibility_timeout_seconds" {
  description = "Visibility timeout in seconds"
  type        = number
  default     = 30
}
```

infra/envs/dev/dev.tfvars

```
queue_name = "exercise-queue-dev"
```

infra/envs/dev/backend-dev.hcl

```
bucket = "YOUR_BUCKET_NAME"
key     = "exercise-7-2/dev/terraform.tfstate"
region  = "us-east-1"
```

infra/envs/staging/staging.tfvars

```
queue_name              = "exercise-queue-staging"
visibility_timeout_seconds = 60
```

infra/envs/staging/backend-staging.hcl

```
bucket = "YOUR_BUCKET_NAME"
key     = "exercise-7-2/staging/terraform.tfstate"
region  = "us-east-1"
```

[.github/workflows/terraform-cd.yml](#) (starter — rewrite this file)

```
name: Terraform CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  terraform-ci:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: infra
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3

      - name: Terraform fmt
        run: terraform fmt -check

      - name: Terraform init
        run: terraform init -backend-config=envs/dev/backend-dev.hcl
        env:
          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
          AWS_REGION: ${ secrets.AWS_REGION }

      - name: Terraform validate
        run: terraform validate

      - name: Terraform plan
        run: terraform plan -var-file=envs/dev/dev.tfvars -out=tfplan
        env:
          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
          AWS_REGION: ${ secrets.AWS_REGION }
```

Setup

Prerequisites

- A GitHub repository where you have admin access (needed to configure Environments and branch protection)
- Three repository secrets added under Settings → Secrets and variables → Actions: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION
- An S3 bucket in your AWS account to store Terraform state. Replace YOUR_BUCKET_NAME in both backend HCL files with its name.

Repository structure

```
.github/
└─ workflows/
   └─ terraform-cd.yml      ← YOU REWRITE THIS
infra/
├─ provider.tf             ← provided (copy as-is)
├─ main.tf                 ← provided (copy as-is)
├─ variables.tf            ← provided (copy as-is)
├─ envs/
│   └─ dev/
│       ├── dev.tfvars      ← provided (copy as-is)
│       └─ backend-dev.hcl  ← provided (edit bucket name)
│   └─ staging/
│       ├── staging.tfvars   ← provided (copy as-is)
│       └─ backend-staging.hcl ← provided (edit bucket name)
```

Tasks

Task 1 — Create GitHub Environments

In your repository on GitHub, go to Settings → Environments and create two environments:

1. `dev` — no protection rules
2. `staging` — enable Required reviewers and add yourself as a reviewer

Task 2 — Split validation into three named PR jobs

Rewrite `terraform-cd.yml` so the `pull_request` trigger fires three separate jobs, each reporting as an individual status check:

- `terraform-fmt` — checks out, runs `terraform fmt -check` (no init required)
- `terraform-validate` — checks out, runs `terraform init -backend=false` then `terraform validate` (no AWS credentials needed)
- `terraform-plan` — checks out, runs full init with backend config, runs `terraform plan -var-file=envs/dev/dev.tfvars -out=tfplan`, saves plan text with `terraform show -no-color tfplan > plan.txt`, uploads both `tfplan` and `plan.txt` as an artifact named `tfplan-dev` using `actions/upload-artifact@v4`, then posts the plan contents as a pull request comment using `actions/github-script@v7`

Each job must use `actions/checkout@v4` and `hashicorp/setup-terraform@v3`.

Reference for the github-script PR comment step:

```
- name: Post plan comment
  if: github.event_name == 'pull_request'
  uses: actions/github-script@v7
  with:
    script: |
      const fs = require('fs');
      const plan = fs.readFileSync('infra/plan.txt', 'utf8');
      github.rest.issues.createComment({
        owner: context.repo.owner,
        repo: context.repo.repo,
        issue_number: context.issue.number,
        body: '### Terraform Plan - dev\n```\n' + plan + '\n```'
      });
```

Task 3 — Add apply jobs

Add two apply jobs that run only on push to main (not on pull_request):

- **apply-dev:** needs all three validation jobs; declares `environment: dev`; downloads the `tfplan-dev` artifact using `actions/download-artifact@v4`; runs `terraform init -backend-config=envs/dev/backend-dev.hcl` then `terraform apply tfplan` (not a fresh plan — use the artifact)
- **apply-staging:** needs `apply-dev`; declares `environment: staging` (this is what causes GitHub to pause and request approval); downloads the same artifact; runs `terraform init -backend-config=envs/staging/backend-staging.hcl` then `terraform apply -var-file=envs/staging/staging.tfvars`

Task 4 — Open a PR and verify

1. Create a branch, make a trivial change (e.g., add a comment to `main.tf`), and push it
2. Open a pull request targeting `main`
3. Confirm that three individual status checks appear on the PR: `terraform-fmt`, `terraform-validate`, `terraform-plan`
4. Confirm that the `terraform-plan` job posts a plan comment on the PR
5. Merge the PR — the `apply-dev` job runs automatically; the `apply-staging` job pauses for your approval
6. Approve the staging deployment and confirm it completes

Save the PR URL as your evidence — paste it into a file called `evidence/pr-url.txt` at the root of your repository.

Acceptance Criteria

- `terraform-cd.yml` defines exactly 5 jobs: `terraform-fmt`, `terraform-validate`, `terraform-plan`, `apply-dev`, `apply-staging`
- The submitted PR shows three individual status checks, each reporting separately
- The `terraform-plan` job posts a plan comment on the PR
- The `apply-dev` job uses `actions/download-artifact` to consume the plan artifact — it does not re-run `terraform plan`
- `apply-staging` declares `environment: staging` and required a manual approval before running
- `evidence/pr-url.txt` exists at the repository root and contains the PR URL where the plan comment is visible